

计算机科学与技术学院神经网络与深度学习课程实验报告

实验题目：RNN-Transformer 实现文本分类		学号：202300130183
日期：2025/11/19	班级：2023 级智能班	姓名：宋浩宇
Email: zhitian0420@gmail		
实验目的： 1. 理解循环神经网络（RNN）和 Transformer 的基本原理和结构。 2. 掌握使用 RNN 和 Transformer 模型对文本数据进行分类的方法。 3. 学习如何评估 RNN 和 Transformer 模型的性能，并调整网络参数以优化性能。 4. 掌握中英文文本数据的预处理方法和特征提取技术。		
实验软件和硬件环境： 软件环境： 主系统：Windows 11 家庭中文版 23H2 ; ArchLinux 编辑器：Visual Studio Code ; PyCharm 2025.1.2 Python 解释器：Python 3.9 Pytorch 版本：2.8.0 CUDA 版本：12.8.97 硬件环境： CPU：13th Gen Intel(R) Core(TM) i9-13980HX 2.20 GHz 内存：32.0 GB (31.6 GB 可用) 磁盘驱动器：NVMe WD_BLACKSN850X2000GB 显示适配器：NVIDIA GeForce RTX 4080 Laptop GPU CUDA 核心数：7424		
实验原理和方法： 实验原理就是 LSTM 和 Transformer 的原理，方法为使用现有数据集划分训练集和测试集，然后跑完训练后再用手写样例测试效果。		
实验步骤：（不要求罗列完整源代码） 1. 先准备数据，因为我电脑上的 torch 环境并不支持 torchtext（版本又莫名其妙的冲突），所以直接上官网手动下载数据集，包括用于英文测试的 AG_NEWS 数据集和用于中文测试的今日头条新闻数据集。 2. 进行数据预处理，主要是分词和 vocabe 划分 <pre># 分词器(简单空格分词) tokenizer = lambda x: x.lower().split() # 构建词汇表 def build_vocab(filepath, max_vocab=VOCAB_SIZE): counter = Counter() df = pd.read_csv(filepath, names=["label", "title", "description"])</pre>		

```

for text in df["title"] + " " + df["description"]:
    counter.update(tokenizer(str(text)))
vocab = {"<pad>": 0, "<unk>": 1}
for word, _ in counter.most_common(max_vocab - 2):
    vocab[word] = len(vocab)
return vocab

```

3. 对于 RNN 任务，构建 LSTM 模型

```

# 模型:LSTM
class LSTMClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim, hidden_dim,
output_dim):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim,
padding_idx=0)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True)
        self.fc = nn.Linear(hidden_dim, output_dim)
    def forward(self, x):
        embedded = self.embedding(x)
        _, (hidden, _) = self.lstm(embedded)
        return self.fc(hidden.squeeze(0))

```

4. 对于 Transformer 任务，构建模型

```

class TransformerCls(nn.Module):
    def __init__(self, vocab_size, d_model, nhead, num_layers, dim_ff,
num_classes):
        super().__init__()
        self.embed = nn.Embedding(vocab_size, d_model, padding_idx=0)
        # 位置编码: 直接利用 nn.Parameter 训练式位置向量(可换
sin/cos)
        self.pos = nn.Parameter(torch.empty(1, MAX_LEN, d_model))
        nn.init.uniform_(self.pos, -0.1, 0.1)
        # TransformerEncoderLayer + Encoder
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=d_model,
            nhead=nhead,
            dim_feedforward=dim_ff,
            dropout=0.1,
            activation='relu',
            batch_first=True # 输入 [B, T, D]
        )
        self.transformer = nn.TransformerEncoder(encoder_layer,
num_layers=num_layers)
        self.fc = nn.Linear(d_model, num_classes)

```

5. 然后用相同的代码做训练就行了

6. 训练完了再做一下测试即可
7. 因为训练实在是太太太太太慢了，所以又加了一个存储和加载模型的函数，节省之后测试的时间。

```
# ----- 保存 -----
def save_model(model, filepath=model_path):
    """
    保存模型权重 + 超参 + vocab, 一站式打包
    """
    torch.save({
        'model_state_dict': model.state_dict(),
        'vocab': vocab, # 前面全局变量
        'model_hparams': {
            'vocab_size': len(vocab),
            'd_model': D_MODEL,
            'nhead': N_HEAD,
            'num_layers': NUM_LAYERS,
            'dim_ff': DIM_FF,
            'num_classes': NUM_CLASSES
        }
    }, filepath)
    print(f"[INFO] Model saved -> {filepath}")

# ----- 加载 -----
def load_model(filepath=model_path, device=DEVICE):
    """
    从磁盘恢复模型(含权重), 返回已.eval()的模型
    """
    if not os.path.isfile(filepath):
        raise FileNotFoundError(filepath)
    checkpoint = torch.load(filepath, map_location=device)
    # 重建模型骨架
    hp = checkpoint['model_hparams']
    model = TransformerCls(
        vocab_size=hp['vocab_size'],
        d_model=hp['d_model'],
        nhead=hp['nhead'],
        num_layers=hp['num_layers'],
        dim_ff=hp['dim_ff'],
        num_classes=hp['num_classes']
    ).to(device)
    model.load_state_dict(checkpoint['model_state_dict'])
    model.eval() # 推断模式
    print(f"[INFO] Model loaded <- {filepath}")
    return model, checkpoint['vocab'] # 同时返回 vocab, 方便 predict
```

另外，因为中文的那个数据长得跟这个不一样，所以需要另外重写一下数据读入的部分。

```
import jieba
df = pd.read_csv('./data/toutiao_news_dataset.txt', sep='!_',
header=None,
names=['id', 'label_id', 'text'], usecols=[0,1,2],
dtype=str, encoding='utf-8')

df = df.sample(n=N_SAMPLE, random_state=SEED).reset_index(drop=True)
label2id = {l:i for i,l in enumerate(df['label_id'].unique())}
id2label = {i:l for l,i in label2id.items()} # 可选:打印用
label_name = {'100' : "民生", '101' : "文化", '102' : "娱乐", '103' :
"体育", '104' : "财经", '106' : "房产", '107' : "汽车", '108' : "教
育", '109' : "科技", '110' : "军事", '112' : "旅游", '114' : "证券",
'115' : "农业", '116' : "电竞"}
tokenizer = lambda x: list(jieba.cut(x, cut_all=False))
```

结论分析与体会：

结果如下：

对于英文：

```
if __name__ == "__main__" and os.path.exists(model_path):
    model, vocab = load_model()
    ex_text_str = "MEMPHIS,Tenn.Four days ago, Jon Rahm was enduring the season's worst weather conditions on Sunday at The \
    Open on his way to a closing 75 at Royal Portrush, which \
    considering the wind and the rain was a respectable showing. \
    Thursdays first round at the WGC - FedEx St.Jude Invitational was another story.With temperatures in the mid - 80 s and hardly any \
    wind, the Spaniard was 13 strokes better in a flawless round. \
    Thanks to his best putting performance on the PGA Tour, Rahm \
    was finished with an8 - under 62 for a three-stroke lead, which \
    was even more impressive considering hed never played the \
    front nine at TPc Southwind."
    # model = model.to("cpu")
    print("This is a %s news" % ag_news_label[predict(ex_text_str)])
```

然后 LSTM 的结果是：

```
[INFO] LSTM model loaded <- lstm_agnews.pt
This is a Sports news
```

Transformer 的结果是：

```
[INFO] Model loaded <- transformer_agnews.pt
F:\Homework\认知科学与类脑计算作业\venv\Lib\site-
.jagged when constructing a nested tensor, a
C:\actions-runner\work\pytorch\pytorch\pyto
output = torch._nested_tensor_from_mask(
This is a Sports news
```

莫名的报错请忽略。

对于中文：

```
if __name__ == "__main__" and os.path.exists(model_path):
    model, vocab = load_model()
    text = "山东大学宋浩宇研制出了能够对中文新闻进行分类的模型！"
    print("这是 %s 新闻" % label_name[id2label[predict(text)]])
```

LSTM 的结果是：

这是 体育 新闻

Transformer 的结果是：

这是 科技 新闻

从结果上来看

英文分类两种方法都挺好的，中文分类 LSTM 效果一般，但是 Transformer 还是可以完成任务的。

就实验过程中遇到和出现的问题，你是如何解决和处理的，自拟 1—3 道问答题：

问题一：训练过程太慢怎么办？

回答一：训练过程太慢可以用 cuda 来完成计算，与之前的方法不同可以在短时间内得到结果，现在做的这些得把训练好的模型存储下来，并在下次的时候直接读取训练好的参数来看效果。